Zewail City of Science, Technology and Innovation

University of Science and Technology

School of Computational Sciences and Artificial Intelligence

CSAI 203 - Fall 2025

Introduction to Software Engineering

Implementation Phase Report

Project: Clinexa Medical System

Version 1.0

Team Number: 26

Email: s-ahmed.ragab@zewailcity.edu.eg

Team Members

Name	**ID**	**Email**
Ahmed Ragab	202400437	s-ahmed.ragab@zewailcity.edu.eg
Mosab Kamal	202402297	s-mosab.gabal@zewailcity.edu.eg
Mohamed Maher	202301798	s-mohammed.kafafy@zewailcity.edu.eg
Abdelrhman Khaled	202400161	s-abdelrhman.abdelhamid@zewailcity.edu.eg

Contents

1 Introduction **2**

2 Developer Guide **2**

2.1 Prerequisites 2

2.2 Installation Steps 2

2.3 Running the Application 3

2.4 Project Structure 3

3 MVC Architecture Implementation **3**

3.1 Model Layer 3

3.2 View Layer 4

3.3 Controller Layer 4

4 Detailed Workflow Example: User Login **4**

5 Project Structure **5**

6 Applied Design Patterns **6**

6.1 1. Repository Pattern 6

6.2 2. Singleton Pattern 7

6.3 3. Decorator Pattern 7

6.4 4. Factory Method Pattern (Implicit) 7

7 Site Handling Functionality **7**

7.1 Data Management 7

7.2 User Workflow Handling 8

7.3 Implementation Delivery 8

1 Introduction

This document serves as the report for the Implementation Phase of the Clinexa Medical System project. It outlines the technical details of the implementation, including the development environment setup, the application of the Model-View-Controller (MVC) architectural pattern, and the operational handling of the site. This phase focuses on delivering a functional web-based healthcare management system allowing interaction between patients and a primary care doctor.

2 Developer Guide

This section provides instructions for setting up the development environment and running the Clinexa application.

2.1 Prerequisites

- **Python 3.x**: Ensure Python is installed on your system.
- **pip**: Python package installer.

2.2 Installation Steps

1. **Clone the Repository**:

```
git clone https://github.com/r4g4b/Clinexa
cd Clinexa
```

2. **Install Dependencies**: The project requires Flask and other standard libraries. Install them using pip:

```
pip install flask
```

2.3 Running the Application

To start the development server, execute the following command in the project root directory:

```
python app.py
```

The application will be accessible at `http://localhost:5000`.

2.4 Project Structure

- **app.py**: The entry point of the application and route definitions.
- **controllers/**: Contains business logic and controller classes.
- **models/**: Defines data structures (Patient, Doctor, Appointment).
- **repositories/**: Handles data persistence to CSV files.
- **templates/**: HTML files for the views (Jinja2).
- **static/**: CSS and assets.
- **data/**: Database files (CSV format).

3 MVC Architecture Implementation

The Clinexa system strictly follows the Model-View-Controller (MVC) design pattern to ensure separation of concerns, maintainability, and scalability.

3.1 Model Layer

The **Model** represents the data and business logic of the application. In Clinexa, models are Python classes located in the `models/` directory.

- **Key Models**: `Patient`, `Doctor`, `Appointment`, `Feedback`, `User`.
- **Data Persistence**: The Repository pattern is used to abstract file I/O operations. Repositories (e.g., `PatientRepository`) in `repositories/` handle reading from and writing to CSV files in the `data/` directory.

3.2 View Layer

The **View** is responsible for presentation and user interaction. Clinexa uses Flask's Jinja2 templating engine, with HTML files located in the `templates/` directory.

- **Dynamic Content**: Views receive data objects (e.g., Doctor object, list of Appointments) from controllers and render them dynamically.
- **Key Templates**:
  - `dashboard.html`: Main hub for patient actions.
  - `booking.html`: Combined view for doctor info, booking form, and reviews.
  - `appointments.html`: Lists upcoming and past appointments.

3.3 Controller Layer

The **Controller** manages the flow of data between the Model and the View. It handles user input, processes it, and updates the Model or View accordingly.

- **Implementation**: Controllers are defined in the `controllers/` directory (e.g., `PatientController.py`, `DoctorController.py`).
- **Routing**: `app.py` acts as the front controller, mapping HTTP requests (GET/-POST) to specific controller methods.
- **Business Logic**: Controllers contain logic for tasks like validating booking requests, calculating new appointment IDs, and coordinating notifications.

4 Detailed Workflow Example: User Login

To illustrate the interaction between MVC components, we detail the **Login** process:

1. **View**: The user accesses the login page (`login.html`). They enter their credentials and submit the form.
2. **Controller**: The request is routed to `app.py`'s `/login` route, which calls the `AuthController.login(email, password)`.

3. **Model Interaction**:
    - The `AuthController` uses the `UserRepository` to search for a user with the provided email.
    - It retrieves the `User` model object.
4. **Logic & Validation**:
    - The controller hashes the input password and compares it with the stored hash in the `User` model using a secure hashing algorithm (SHA-256).
    - If valid, a session is created. If invalid, the controller increments a failure counter (implementing the account locking security requirement).
5. **View Update**:
    - **Success**: The user is redirected to `dashboard.html`.
    - **Failure**: The user is returned to `login.html` with a flash error message.

5 Project Structure

The following directory tree represents the codebase organization, highlighting the modularity of the system:

```
Clinexa/
|-- app.py                  # Application entry point & Routing
|-- controllers/            # Business Logic Layer
|   |-- auth_controller.py
|   |-- patient_controller.py
|   |-- doctor_controller.py
|   |-- ...
|-- models/                 # Data Models
|   |-- user.py
|   |-- patient.py
|   |-- doctor.py
|   |-- appointment.py
```

```
|   |-- ...
|-- repositories/           # Data Access Layer
|   |-- user_repository.py
|   |-- appointment_rep.py
|   |-- ...
|-- core/                   # Shared Utilities
|   |-- csv_manager.py      # File I/O handler
|   |-- security.py         # Hashing & Validation
|-- templates/              # Presentation Layer (Views)
|   |-- login.html
|   |-- dashboard.html
|   |-- booking.html
|   |-- ...
|-- data/                   # Data Storage (CSV)
|   |-- patients.csv
|   |-- doctors.csv
|-- static/                 # Static Assets (CSS, JS)
```

6 Applied Design Patterns

Beyond the core MVC architecture, the Clinexa system implements several other key design patterns to ensure robust and clean code.

6.1 1. Repository Pattern

Where: `repositories/` directory (e.g., `UserRepository`, `AppointmentRepository`).
How: The application strictly separates business logic from data access. The controllers interact only with Repository classes to perform CRUD operations. These repositories internally handle the parsing of CSV files, mapping raw text data to Python objects. This encapsulation ensures that the application's core logic remains unaware of the underlying CSV storage mechanism, promoting cleaner code and easier testing.

6.2 2. Singleton Pattern

Where: `CSVManager` in `core/csv_manager.py`.

How: The `CSVManager` class is implemented as a Singleton to provide a centralized point of access for all file I/O operations. This is critical for data integrity in a file-based system. It manages file locks to prevent race conditions when multiple concurrent requests attempt to write to the `patients.csv` or `appointments.csv` files simultaneously, ensuring thread-safe operations.

6.3 3. Decorator Pattern

Where: Authentication wrappers in `app.py` (e.g., `@login_required`, `@patient_required`).

How: We utilize Python's `functools.wraps` to create higher-order functions that wrap route handlers. These decorators intercept incoming requests to validate the user's session state and role permissions before the request reaches the controller logic. If validation fails, the decorator redirects the user to the login page or displays an error, effectively enabling cross-cutting concerns like security to be applied reuseably across the application.

6.4 4. Factory Method Pattern (Implicit)

Where: Data instantiation in Repositories.

How: The repositories act as factories for Model objects. When reading from the CSV files, raw string data is parsed and passed to the constructor methods of classes like `Patient` or `Appointment`. This centralizes the object creation logic, ensuring that valid, fully-formed objects are always returned to the application layer, encapsulating the complexity of parsing pipe-separated values.

7 Site Handling Functionality

7.1 Data Management

The site handles data using a file-based CSV storage system. This lightweight approach allows for easy deployment and portability without a heavy database dependency.

- **Concurrency**: The `CSVManager` utilizes threading locks to handle concurrent read-/write operations safely.
- **Repositories**: Dedicated repositories ensure data consistency for Patients, Doctors, and Appointments.

7.2 User Workflow Handling

The application is optimized for a single-doctor clinical setting:

- **Authentication**: Role-based login (Patient/Doctor) routes users to tailored dashboards.
- **Booking Flow**: The booking process is streamlined. Since the system services a single primary doctor, the booking interface automatically selects the doctor, displaying their profile and reviews alongside the scheduling form. This minimizes user friction.
- **Session Management**: Flask Sessions are used to maintain user state securely across requests.

7.3 Implementation Delivery

The project uses Git for version control. Collaboration features of GitHub (Pull Requests, protected branches) were utilized to ensure code quality. Features were developed on separate branches and merged into `main` after review, adhering to the course's delivery guidelines.